

Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

School of Computer Science and Statistics

CSU33D06

Project : analysis of a GitHub project

Chainladder-python

Eloïse Sallet 25387824

Contents

1 Introduction **2**

2 Stakeholders and Context View **2**
2.1 Stakeholders 2
2.2 Context view 3

3 Software Evolution Analysis **4**
3.1 Frequency of Commits 4
3.2 Hotspots Analysis 4

4 Reliability and Collaboration Analysis **5**
4.1 Code Review Velocity : Pull Requests 5

5 Complexity Analysis **6**

6 Conclusion **6**

1 Introduction

Actuarial science has always been a field of great interest to me. This is the application of mathematical and statistical methods to analyse and predict risk in insurance, finance, and other industries. Since it is a path I want to follow both academically and professionally, I chose a project linked to this field.

The project I have chosen is Chainladder-python, a 100% Python open-source library (`https://github.com/casact/chainladder-python`) built by a group of volunteers with more than 1700 commits. It handles all the actuarial needs with different methods by using its own API so users no longer have to rely on outdated softwares and tools like Excel. Chainladder was built initially in R, but this Python extension is easy to manipulate for a user who already worked with Pandas and Scikit-learn libraries.

The repository is organized into a specific structure: the `chainladder/` directory contains the source code, subdivided into `core/` (the engine and data structures), `methods/` (actuarial models), and `utils/` (helper functions). The root also includes essential files for environment management and distribution.

In this report, the goal is to do a deep analysis of the project's design and development dynamics. In my initial approach, I tried to do a dynamic analysis using *line-profiler* and *memory-profiler*. However, these tools did not yield significant bottlenecks to analyze because the library is highly optimized and its execution is often nearly instantaneous for standard actuarial datasets.

So, I focused more about a socio-technical and structural analysis approach to uncover how the system actually lives and evolves. First, we will identify the key actors using the Rozanski and Woods approach for Stakeholders and Context View. Then, we will focus on Software Evolution through commit frequency and Hotspot identification. Next, we will evaluate development efficiency by examining Code Review Velocity in Pull Requests. Finally, we will measure Cyclomatic Complexity to assess the maintainability of the project's most critical files.

2 Stakeholders and Context View

2.1 Stakeholders

Before analysing how the system works and evolves, it is important to identify the stakeholders who work for and with the system. I followed the Rozanski and Woods approach to identify them.

The evolution of the project is handle by a central core group composed of John Bogaardt (the original creator), Kenneth Hsu (the working group chair), Henry Ding, and Gene Dan. They have key roles in the structure. As Maintainers and Assessors, they control the version of the system and make sure every Pull Request follows standard actuarial math before merging. Moreover, they act as Developers and Production Engineers by translating complex actuarial requirements into Python code and managing the automation of tests via GitHub Actions across various environments.Communication and outreach are specifically handled by John Bogaardt and Kenneth Hsu. Their roles allow them to bridge the gap between technical code and the actuarial community.

But while this core group leads the project, it is supported by a community of more than 30 developers who contribute to the codebase. Moreover, to maintain coordination, the Contributors Working Group meets every week to discuss current issues, upcoming features, and the direction of the system's design.

There is also the Casualty Actuarial Society (CAS) who is the primary Acquirer. They provide the institutional infrastructure, legitimacy, and professional network required for the project. The CAS also acts as a Supplier by providing expert volunteers and standardized actuarial datasets, used to build the package.

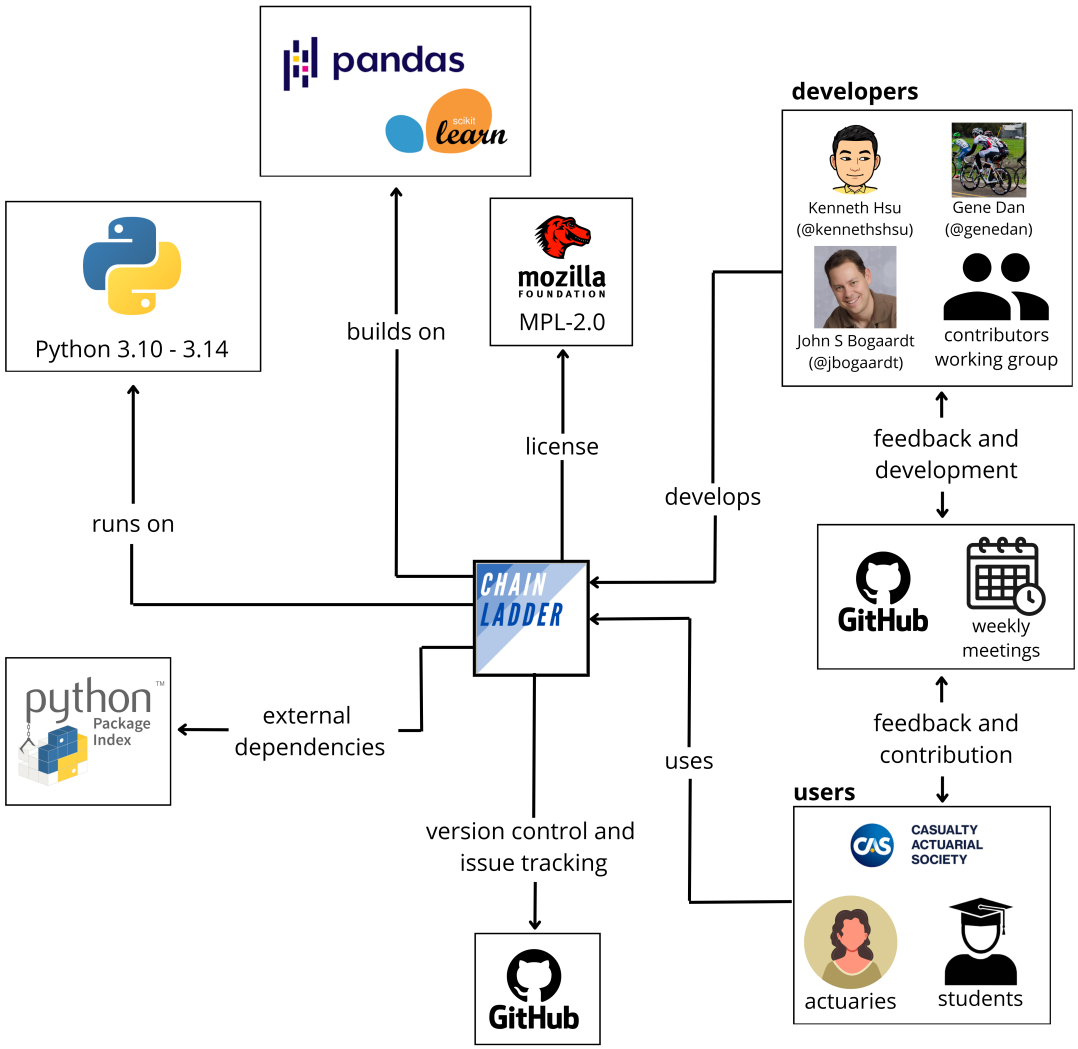
Beyond the human actors, the project relies on many Suppliers. There are the Python Software Foundation, Pandas, and Scikit-learn that provide the fundamental building blocks. But also GitHub and PyPI who serve as the distribution platforms. Support and testing are shared between the official Working Group and the user community via GitHub Discussions. Moreover, the community often acts

as beta-testers by reporting bugs found in real-world insurance data. The project also uses Dependabot as an automated stakeholder to monitor and manage dependencies.

This significant overlap of roles where the same individuals define, build, and validate the system allows a consistent vision and rapid decision-making. However, it can also means that the project's sustainability relies on a few people.

2.2 Context view

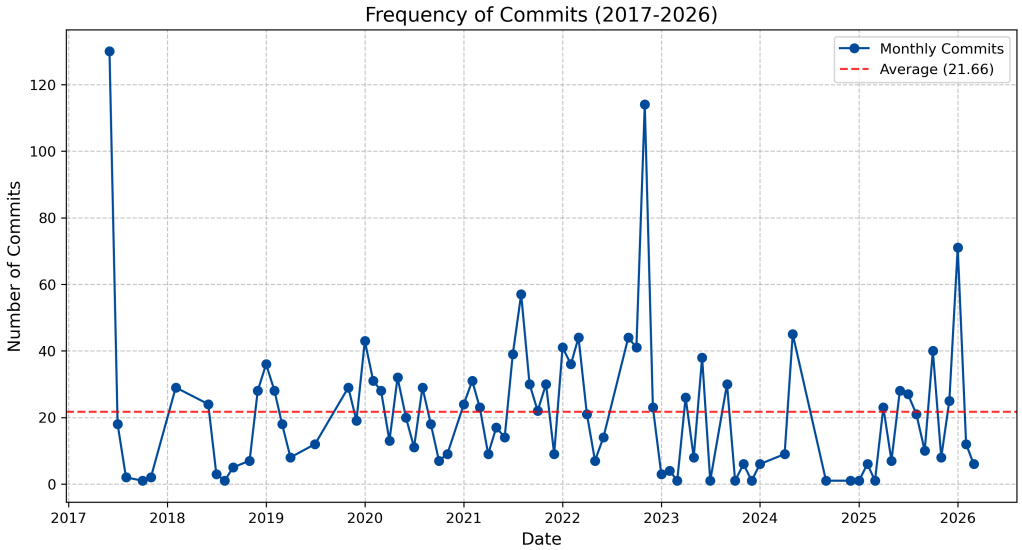
According to Rozanski and Woods, a context view point describes the relationships, dependencies, and interactions between the system and its environment (the people, systems, and external entities with which it interacts). In the case of Chainladder, I mapped the system to show how it fits into the Python environment. The library works as a link between general data tools like Pandas or NumPy and the specific needs of actuaries. The diagram shows that the system depends on external platforms like GitHub for its development and PyPI for its distribution.



3 Software Evolution Analysis

3.1 Frequency of Commits

To understand how the project evolves over time, I analysed the frequency of commits. I made a Python script using the `pydriller` library to extract the commit history. Since there are a lot of commits (almost 1800), I decided to do an history plot.



This plot reveals a kind of stable activity, with an approximately average of 21 commits per month, which indicates consistent maintenance. However some key phases can be identified:

First, there is a high volume of commits at the very beginning of the project. This is expected because it represents the initial push to establish the core architecture of the project.

Second, a major peak of 106 commits is visible in November 2022. It is directly linked to the CAS Annual Meeting. This was a critical moment where John Bogaardt and Kenneth Hsu presented Chainladder to the professional community for the first time. The intense activity during this month was focused on creating a 'zero-installation' experience, requiring a big review of the documentation and the integration of training workbooks.

Finally, we can observe another period in January 2026 where the number of commits increase. This corresponds to the release of Version 0.9.0, a major update that introduced support for Python 3.14, with a big review on documentation, and work on dependency issues.

These peaks illustrates how external professional events and major version milestones directly influence the software's design and evolution. Beyond those, the regularity of commits confirms the project's maturity and the core team's commitment to long-term maintainability.

3.2 Hotspots Analysis

In order to analyse the maintainability of the system, it is important to first observe which files can cause more problems, so I performed a Hotspot analysis. A Hotspot is a file that undergoes frequent modifications, which often indicates either high architectural importance or high complexity requiring constant maintenance.

Initially, I did an analysis by filename with my `pydriller` script. But, the results were biased as multiple files shared the same name across different modules. So, I used full file paths with the same `pydriller` script. By disambiguating these paths, the true hierarchy of the system's evolution emerged, as shown in the table below.

File Path	Modifications	Role in the System
`chainladder/core/triangle.py`	199	Primary Data Structure
`chainladder/core/base.py`	180	Core Estimator Base Logic
`chainladder/core/tests/test_triangle.py`	132	Quality Gate (Unit Testing)
`setup.py`	123	Dependency Management
`chainladder/core/pandas.py`	109	Pandas Integration Layer

We can observe that `chainladder/core/triangle.py` is the actual central hotspot of the project. That is not surprising as in actuarial science, a "Triangle" is the standard way of organizing insurance data to track how claims develop over time. This is the digital foundation of the library, so this file must be extremely robust. Its high modification count, closely followed by its test file (`test_triangle.py`), suggests a continuous effort to improve data validation and support new actuarial formats.

Furthermore, by using the path, I understood more how the project is built. The file `base.py` exists in 3 different folders. This discovery proves that the system follows a Modular Design. Indeed, instead of a single big file, the project distributes its logic across specialized modules, even if they share similar naming conventions for consistency.

Finally, the presence of `setup.py` confirms the project's high maintenance level, ensuring compatibility with the evolving Python ecosystem and managing the various dependencies required for modern actuarial workflows.

4 Reliability and Collaboration Analysis

4.1 Code Review Velocity : Pull Requests

Now, let's focus on velocity which is the speed at which code changes move through the review process, from the initial Pull Request (PR) submission until it is approved and merged. This metric is a crucial socio-technical indicator, as it tracks how quickly the development team can incorporate new features and bug fixes into the codebase while maintaining quality gates.

The analysis of the chainladder repository reveals a flexible global workflow where the review rigor scales with the complexity of the change. I observed that the duration from opening to merging varies significantly based on the nature of the contribution:

- Automated PRs (such as those from *Dependabot*) and minor dependency updates are often resolved in a matter of hours. This indicates a high level of trust in the Continuous Integration (CI) pipeline, which acts as an automated quality gate.
- For major architectural changes or complex actuarial features, the velocity is intentionally moderated. For example, PR #671 required almost 30 days of discussion to ensure alignment with professional API standards.

To better understand the review process, I analysed several key Pull Requests that represent the diverse nature of the system's evolution:

- Collaborative Expertise (#643): This PR for a specific method involved a deep technical debate with over 20 comments between authors and subject-matter experts. Despite the presence of automated tests, the team prioritized knowledge sharing and human expertise over pure integration speed, ensuring the math remains compliant with actuarial standards.
- Governance and Decision Making (#650): A proposal for a new automated linting solution was rejected despite its technical merits. The maintainers chose to prioritize community accessibility and a low barrier to entry for new actuarial contributors over technical perfection. This proves that stakeholder needs directly dictate the velocity and design choices of the project.

The code review velocity illustrates a balance between automated efficiency and expert human review. While the Contributors Working Group demonstrates high responsiveness for routine maintenance, they act as strict Assessors for core logic. This ensures that the high velocity of the Expansion Phase observed in 2026 does not compromise the architectural health or the reliability of the system under test (SUT).

5 Complexity Analysis

To evaluate the maintainability and architectural health of the system, I used `lizard` in order to obtain the Cyclomatic Complexity (CC). I focused on the core files previously identified as Hotspots.

The overall health of the `chainladder/core` directory is very good, with most files maintaining an Average Cyclomatic Complexity (AvgCCN) between 3.0 and 5.0. This means that the functions are generally short and easy to test.

File Path	NLOC	AvgCCN	Warnings	Status
`core/base.py`	471	3.4	0	Highly Maintainable
`core/triangle.py`	711	5.0	2	Complex Core
`core/dunders.py`	321	3.4	3	High Complexity Zones
`core/tests/test_triangle.py`	728	1.1	0	Excellent (Simple Tests)

While the averages are healthy, the `lizard` tool triggered several warnings for specific functions that exceed the standard threshold of 15 where the most complex logic resides.

The file `triangle.py` contains two complex functions: `__init__` (CCN 21) and `grain` (CCN 25). The high complexity of the constructor is justified by the need to validate and ingest various data formats (Pandas, Numpy, Polars) while ensuring actuarial consistency. Similarly, the `grain` function must reconcile different time frequencies, which inherently requires dense conditional logic.

Interestingly, the `dunders.py` file (which handles Python arithmetic operation like addition and subtraction) contains the highest individual complexity peaks, such as `_prep_origin_development` with a CCN of 20. This is a critical design choice: the complexity is centralized in this "utility" file to ensure that actuarial triangles can be manipulated with simple arithmetic operators ($+$, $-$, $*$, $/$), providing a seamless experience for the user while hiding the underlying complexity of aligning non-square matrices.

I believe the average complexity remains low because the developers have effectively distributed the logic across a large number of specialized files. The project relies on a high volume of small, focused files and folders. This encapsulation ensures that the project remains maintainable for the Core Team while being accessible to the 30+ developers and the broader CAS community. This architectural trade-off keeps the "User-Facing" files and test suites (like `test_triangle.py` with a CCN of 1.1) extremely simple to encourage community contributions.

6 Conclusion

This analysis of Chainladder-python highlights a robust open-source project that bridges the gap between traditional actuarial methods and modern software engineering. By pivoting from a purely technical profiling to a socio-technical approach, I was able to uncover the true strengths of the system.

The data extracted via *pydriller* and *lizard* confirms that the project is built on a healthy foundation. Furthermore, the high code review velocity and the active involvement of the CAS Contributors Working Group demonstrate that the project's sustainability is secured by a professional and responsive community.

In conclusion, Chainladder-python is more than just a library, it is a successful digital transformation of actuarial science. Its design choices prioritize long-term maintainability and community accessibility, making it a reliable industry standard for the years to come.